

Uso de la pila en el código:

1. Postergación de Operadores: En la notación posfija, los operadores deben ir estrictamente *después* de sus operandos (por ejemplo, ab+). Cuando el código escanea un operador como +, no puede escribirlo en el resultado de inmediato porque aún no conoce el siguiente operando. La pila sirve para mantener ese operador en espera.
2. Ordenamiento por Jerarquía (LIFO): Al ser una estructura donde lo último en entrar es lo primero en salir, permite apilar operadores por niveles de prioridad. Si un operador de mayor prioridad (como *) se coloca encima de uno de menor prioridad (como +), se garantiza que al limpiar la pila, el de mayor peso salga primero hacia el resultado.
3. Control de Paréntesis: Los paréntesis rompen las reglas matemáticas normales. Al meter un (a la pila, este actúa como un "piso falso" o barrera. Ningún operador que esté debajo de él puede salir hasta que aparezca el) de cierre, lo que fuerza a que todo lo que esté adentro se resuelva primero.

Cómo interactúa la pila con el código proporcionado

Expresión: $a + b * c$

- Lectura de a: El código detecta que `char.isalnum()` es verdadero. El operando a pasa directo a la lista del resultado.
 - *Resultado:* [a] | *Pila:* [Vacía]
- Lectura de +: La pila está vacía, por lo que el ciclo while de comparación no se ejecuta. El código hace `operator_stack.push('+')`.
 - *Resultado:* [a] | *Pila:* [+]
- Lectura de b: Es un operando, va directo al resultado.
 - *Resultado:* [a, b] | *Pila:* [+]
- Lectura de *: El código evalúa las prioridades en el while:

Python

```
prioridad_cima = self.precedence.get(cima, 0) # Obtiene 1 (de '+')
prioridad_actual = self.precedence[char]     # Obtiene 2 (de '*')
if prioridad_cima >= prioridad_actual:      # ¿1 >= 2? FALSO
```

Como la condición es falsa, el ciclo se rompe sin sacar nada. El código ejecuta `operator_stack.push('*')`, quedando el operador de mayor prioridad arriba.

- *Resultado:* [a, b] | *Pila:* [+, *] (* está en el `.top()`)
- Lectura de c: Va directo al resultado.
 - *Resultado:* [a, b, c] | *Pila:* [+, *]
- Fin de la cadena: El texto se termina y se activa el último bloque while para vaciar la pila:

Python

```
while not operator_stack.is_empty():
```

```
    postfix_result.append(operator_stack.pop())
```

El método `.pop()` extrae primero el elemento en la cima (*) y luego el restante (+).

- *Resultado Final:* abc*+

Expresión: (a + b) * c

- Lectura de (: El código entra directamente en la condición `elif char == '('`: y ejecuta `operator_stack.push('(')`.
 - *Resultado:* [] | *Pila:* [(]
- Lectura de a: Al ser un operando, se añade al resultado.
 - *Resultado:* [a] | *Pila:* [(]
- Lectura de +: Se evalúa en el bloque de operadores. Como la cima de la pila es (, el código rompe el bucle de comparación y hace `operator_stack.push('+')`. El paréntesis protege al operador.
 - *Resultado:* [a] | *Pila:* [(, +]
- Lectura de b: Va directo al resultado.
 - *Resultado:* [a, b] | *Pila:* [(, +]
- Lectura de): Al detectar el cierre, se activa el bloque de código encargado de limpiar el paréntesis:

Python

```
elif char == ')':
```

```
while not operator_stack.is_empty() and operator_stack.top() != '(':
```

```
    postfix_result.append(operator_stack.pop())
```

Este ciclo extrae obligatoriamente todo lo que se encuentre arriba del (, provocando que el + salga de la pila y se mueva al resultado inmediatamente. Después, la línea siguiente ejecuta un .pop() adicional para remover y desechar el (de la pila.

- *Resultado:* [a, b, +] | *Pila:* [Vacía]
- Lectura de *: La pila está vacía, por lo que se guarda con .push('*').
 - *Resultado:* [a, b, +] | *Pila:* [*]
- Lectura de c: Se añade al resultado.
 - *Resultado:* [a, b, +, c] | *Pila:* [*]
- Fin de la cadena: El bucle de vaciado final extrae el * que quedaba en la estructura.
 - Resultado Final: ab+c*